

Міністерство освіти і науки України

Харківський національний університет радіоелектроніки

Факультет Комп'ютерних наук

Кафедра Програмної інженерії

ЛАБОРАТОРНА РОБОТА № 3

з дисципліни

«АНАЛІЗ ДАНИХ НА ОСНОВІ ШТУЧНОГО ІНТЕЛЕКТУ»

підготовки бакалавра

спеціальності 121 «Інженерія програмного забезпечення»

спеціалізації «Програмна інженерія»

Виконав:

студент гр. ПЗПІ-23-8

_____ Тарасов Р.М.

_____ Потурайко Н.І.

Перевірив:

к.т.н., доц. кафедри ПІ

_____ Работягов А.В.

Оцінка:

«1» грудня 2025 р.

«____» _____ 2000 р.

Лабораторна робота № 3.

1. Мета роботи

Розробка технічної системи автоматичної бінарної класифікації мовного сигналу за критерієм «Людина – Штучний інтелект (ШІ)» з використанням біонічного методу розпізнавання.

2. Технічні засоби

- Середовище розробки: Python (PyCharm);
- Бібліотеки: numpy, scipy, matplotlib;
- ПЗ для обробки аудіо: MAGIX Sound Forge Pro Suite 18.

3. Хід виконання роботи

Етап 1. Формування бази даних мовних сигналів

Було сформовано два класи сигналів:

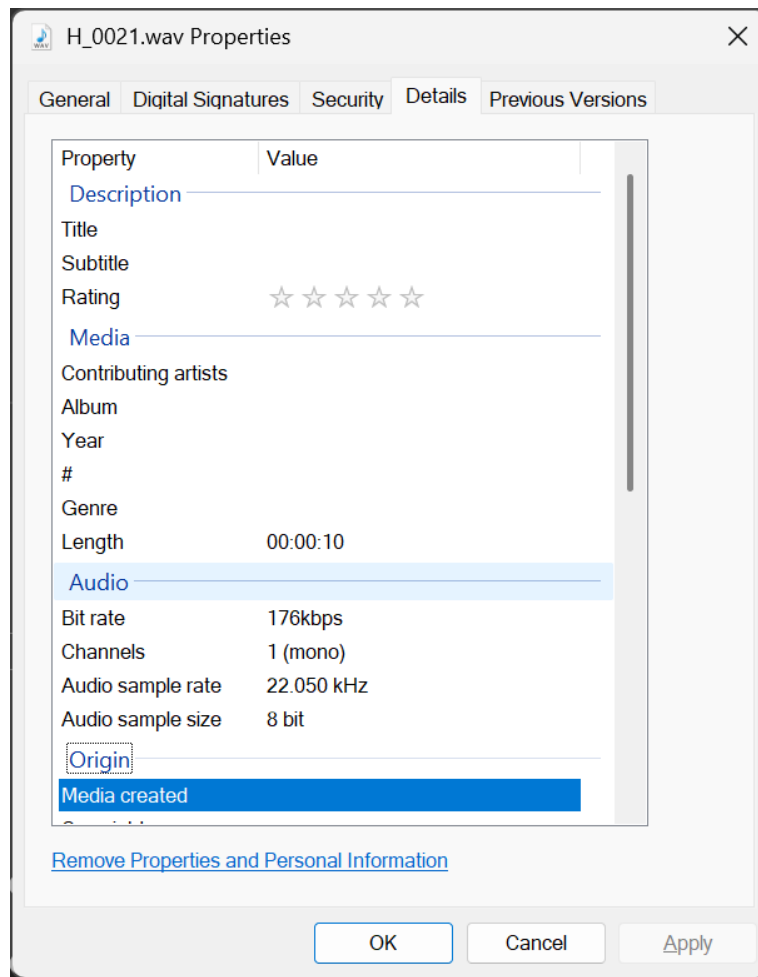
Клас «Людина» (H): власний голосовий запис (українська мова) та зразки з відкритих джерел.

Клас «ШІ» (AI): синтезоване мовлення, отримане за допомогою сучасних нейромережових генераторів (TTS) різними мовами.

Етап 2. Попередня обробка сигналів (Sound Forge)

Для уніфікації даних та виконання вимог методичних вказівок усі файли пройшли пакетну обробку (Batch Processing) з такими параметрами:

- Частота дискретизації: 22050 Hz;
- Кількість каналів: Mono;
- Розрядність: 8-bit (без дитерінгу);
- Тривалість сегментів: 10 секунд (нарізка скриптом).



Етап 3. Програмна реалізація біонічного методу

Розроблено скрипт на мові Python, який реалізує такий алгоритм:

Сегментація: розбиття сигналу на ділянки аналізу $T = 2500$ семплів.

Побудова трикутника голосних: виділення ознак $v1, v2$ (на основі коду ЛР №1).

Пошук щільності: реалізовано алгоритм «ковзного вікна» для пошуку 3-х центрів максимальної щільності точок згідно з формулою:

$$r(W) = \sum_p \left(\frac{1}{l \cdot h} \cdot \frac{1}{n(p)} \sqrt{\sum |p^i - p^j|^2} \right)$$

Розрахунок відстаней: Обчислення метрики R (відстані між центрами кластерів) за формулою Евкліда.

Етап 4. Аналіз результатів (Графічна інтерпретація)

Було проведено розрахунок значень R для всієї вибірки (12561 значення для класу Н та 3222 значень для класу АІ). Побудовано графік ранжованого розподілу значень.



Аналіз розподілу:

Графік демонструє, що на нормованому інтервалі (від 30% до 90% вибірки) крива класу «Людина» (синя) стабільно знаходиться вище кривої класу «ШІ» (червона).

Клас «Людина»: Характеризується більшими значеннями R та плавнішим зростанням. Це пояснюється природною варіативністю артикуляції та наявністю шумів, що призводить до більшого розкиду точок у фонетичному просторі трикутника Хелвага-Щерби.

Клас «ШІ»: Демонструє менші значення R (компактніші кластери) на основній ділянці, що свідчить про стабільнішу, «математично ідеальну» генерацію фонем без природних артикуляційних відхилень.

4. Висновки

У ході лабораторної роботи було розроблено систему бінарної класифікації мовних сигналів.

На основі отриманих експериментальних даних сформовано вирішальне правило:

Для 8-бітних сигналів частотою 22050 Гц, якщо середнє значення відстані між центрами кластерів R_{avg} на тестовій ділянці перевищує поріг 16, сигнал класифікується як «Людина». В іншому випадку ($R \leq 16$) – як «Штучний інтелект».

Код програми:

```
import dash
from dash import dcc, html, Input, Output, State, no_update
import dash_bootstrap_components as dbc
import plotly.graph_objs as go
import numpy as np
from pathlib import Path
from scipy.io import wavfile
from scipy.signal import find_peaks
from scipy.spatial.distance import pdist
from typing import Tuple, Optional, List

FINAL_DATA_DIRECTORY = r""

SAMPLE_RATE = 22050
T_ANALYSIS = 2500
SMOOTHING_WINDOW = 16
PEAK_PROMINENCE = 10
L_RANGE = (80, 200)

L_MAX = 200
H_MAX = 200
WINDOW_L = 15
WINDOW_H = 15
STEP_SIZE = 3
MIN_POINTS = 2

def smooth_signal(signal: np.ndarray, window_size: int) -> np.ndarray:
    kernel = np.ones(window_size) / window_size
    smoothed = np.convolve(signal, kernel, mode='same')
    return smoothed

def _find_best_oscillation_in_window(window: np.ndarray, l_range: Tuple[int, int]) -> Optional[Tuple[int, int, int]]:
    valleys, _ = find_peaks(-window, distance=SMOOTHING_WINDOW + 1,
prominence=PEAK_PROMINENCE)
    peaks, _ = find_peaks(window, distance=SMOOTHING_WINDOW + 1, prominence=PEAK_PROMINENCE)
    if len(valleys) < 2 or len(peaks) < 1: return None

    best_oscillation = None
    max_energy = -np.inf
    l_min, l_max = l_range
    for i in range(len(valleys) - 1):
        p0 = valleys[i]
        suitable_peaks = peaks[peaks > p0]
```

```

if len(suitable_peaks) == 0: continue
p1 = suitable_peaks[0]
suitable_valleys_after_peak = valleys[valleys > p1]
if len(suitable_valleys_after_peak) == 0: continue
p2 = suitable_valleys_after_peak[0]
length = p2 - p0
if l_min <= length <= l_max:
    energy = np.sum(window[p0:p2])
    if energy > max_energy:
        max_energy = energy
        best_oscillation = (p0, p1, p2)
return best_oscillation

```

```

def extract_features(smoothed_signal: np.ndarray, window_t: int, l_range: Tuple[int, int]) -> np.ndarray:
    features = []
    current_pos = 0
    while current_pos + window_t < len(smoothed_signal):
        window_data = smoothed_signal[current_pos: current_pos + window_t]
        oscillation_points = _find_best_oscillation_in_window(window_data, l_range)
        if oscillation_points:
            p0, p1, p2 = oscillation_points
            v1, v2 = float(p1 - p0), float(p2 - p1)
            if v1 > 0 and v2 > 0: features.append([v1, v2])
            current_pos += p2
        else:
            current_pos += window_t // 4
    return np.array(features)

```

```

def find_3_density_centers(points: np.ndarray) -> Tuple[List[Tuple[float, float]], List[Tuple[float, float, float]]]:
    candidates = []
    debug_heatmap = []

```

```

for x in range(0, L_MAX - WINDOW_L, STEP_SIZE):
    for y in range(0, H_MAX - WINDOW_H, STEP_SIZE):
        x_min, x_max = x, x + WINDOW_L
        y_min, y_max = y, y + WINDOW_H
        mask = (points[:, 0] >= x_min) & (points[:, 0] <= x_max) & \
            (points[:, 1] >= y_min) & (points[:, 1] <= y_max)
        window_points = points[mask]
        n_p = len(window_points)
        if n_p < MIN_POINTS: continue

        pairwise_dist_sum = np.sum(pdists(window_points, metric='euclidean'))
        r_w = pairwise_dist_sum / (n_p * n_p + 1e-6)

        center_x = x + WINDOW_L / 2
        center_y = y + WINDOW_H / 2
        candidates.append((r_w, center_x, center_y))
        debug_heatmap.append((center_x, center_y, r_w))

```

```

candidates.sort(key=lambda x: x[0])
final_centers = []
for cand in candidates:
    if len(final_centers) >= 3: break
    r, cx, cy = cand
    is_far = True
    for _, ex, ey in final_centers:
        if np.sqrt((cx - ex) ** 2 + (cy - ey) ** 2) < 15:
            is_far = False;
            break

```

```

        if is_far: final_centers.append(cand)

    if len(final_centers) < 3 and len(candidates) >= 3:
        needed = 3 - len(final_centers)
        for cand in candidates:
            if needed == 0: break
            if cand not in final_centers:
                final_centers.append(cand)
                needed -= 1

    return [(c[1], c[2]) for c in final_centers], debug_heatmap


def calculate_R_distances(centers: List[Tuple[float, float]]) -> List[float]:
    if len(centers) < 3: return []
    c1, c2, c3 = centers[0], centers[1], centers[2]
    R12 = np.sqrt((c1[0] - c2[0]) ** 2 + (c1[1] - c2[1]) ** 2)
    R13 = np.sqrt((c1[0] - c3[0]) ** 2 + (c1[1] - c3[1]) ** 2)
    R23 = np.sqrt((c2[0] - c3[0]) ** 2 + (c2[1] - c3[1]) ** 2)
    return [R12, R13, R23]


app = dash.Dash(__name__, external_stylesheets=[dbc.themes.DARKLY])

app.layout = dbc.Container([
    html.H1("ЛР №3: Класифікація 'Людина - ШИ'", className="my-4 text-center"),

    dbc.Card(body=True, className="mb-4", children=[
        dbc.Row([
            dbc.Col([
                html.Label("Робоча директорія:"),
                dbc.Input(value=FINAL_DATA_DIRECTORY, id="input-dir", readonly=True),
            ], width=9),
            dbc.Col([
                dbc.Button("Запустити повний аналіз", id="btn-run-all", color="primary", className="w-100 mt-4"),
            ], width=3)
        ])
    ]),

    dcc.Tabs([
        dcc.Tab(label='Глобальний результат (Рис. 6)', children=[
            dcc.Graph(id='graph-global', style={'height': '70vh'}),
            html.Div(id='global-stats', className="mt-2")
        ]),

        dcc.Tab(label='Інспектор файлів (Детальний аналіз)', children=[
            dbc.Row([
                dbc.Col([
                    html.Label("Клас:"),
                    dcc.Dropdown(id='dd-class', options=[
                        {'label': 'Human (H)', 'value': 'H'},
                        {'label': 'AI (ШИ)', 'value': 'AI'}
                    ], value='H', clearable=False)
                ], width=3),
                dbc.Col([
                    html.Label("Файл:"),
                    dcc.Dropdown(id='dd-file', clearable=False)
                ], width=6),
                dbc.Col([
                    html.Label("Часовий фрагмент (Chunk):"),
                    dcc.Dropdown(id='dd-chunk', clearable=False)
                ], width=3),
            ])
        ])
    ])

```

```

], className="my-3"),

dbc.Row([
    dbc.Col([
        dcc.Graph(id='graph-detail-scatter', style={'height': '60vh'})
    ], width=8),
    dbc.Col([
        html.H5("Параметри фрагмента"),
        html.Div(id='detail-info'),
        html.Hr(),
        dcc.Graph(id='graph-detail-signal', style={'height': '30vh'})
    ], width=4)
])
]),
]),

dcc.Store(id='store-global-data'),
dcc.Store(id='store-file-list')
], fluid=True)

@app.callback(
    [Output('graph-global', 'figure'),
     Output('store-global-data', 'data'),
     Output('store-file-list', 'data'),
     Output('global-stats', 'children')],
    Input('btn-run-all', 'n_clicks'),
    State('input-dir', 'value'),
    prevent_initial_call=True
)
def run_global_analysis(n, base_dir):
    base_path = Path(base_dir)
    class_R_data = {'H': [], 'AI': []}
    file_list = {'H': [], 'AI': []}

    for cls in ['H', 'AI']:
        seg_dir = base_path / f'{cls}_Segments'
        if not seg_dir.exists(): continue
        files = list(seg_dir.glob('*.wav'))
        file_list[cls] = [f.name for f in files]

    for f in files:
        try:
            rate, raw = wavfile.read(f)
            sig = raw.astype(np.float64) - 128.0 if raw.dtype == np.uint8 else raw.astype(np.float64)
            smoothed = smooth_signal(sig, SMOOTHING_WINDOW)
            chunks = max(1, len(smoothed) // T_ANALYSIS)
            for i in range(chunks):
                chunk = smoothed[i * T_ANALYSIS: (i + 1) * T_ANALYSIS]
                pts = extract_features(chunk, 500, L_RANGE)
                if len(pts) >= MIN_POINTS:
                    ctrs, _ = find_3_density_centers(pts)
                    if len(ctrs) == 3:
                        class_R_data[cls].extend(calculate_R_distances(ctrs))
        except:
            pass

    R_H = np.sort(class_R_data['H'])
    R_AI = np.sort(class_R_data['AI'])

    fig = go.Figure()
    if len(R_H) > 0:
        fig.add_trace(

```



```

        go.Scatter(x=np.linspace(0, 100, len(R_H)), y=R_H, name='Human (H)', line=dict(color='blue',
width=3)))
    if len(R_AI) > 0:
        fig.add_trace(go.Scatter(x=np.linspace(0, 100, len(R_AI)), y=R_AI, name='AI (ШІ)',
            line=dict(color='red', width=3, dash='dash'))))

    fig.update_layout(title="Розподіл відстаней R (Нормовано)", xaxis_title="Вибірка (%)",
        yaxis_title="R",
        template="plotly_dark")

    stats = f"Всього значень: H={len(R_H)}, AI={len(R_AI)}. Середнє R_H={np.mean(R_H):.2f},
R_AI={np.mean(R_AI):.2f}"

    return fig, class_R_data, file_list, dbc.Alert(stats, color="info")

@app.callback(
    Output('dd-file', 'options'),
    Input('dd-class', 'value'),
    State('store-file-list', 'data')
)
def update_file_dropdown(cls, files_data):
    if not files_data: return []
    return [{'label': f, 'value': f} for f in files_data.get(cls, [])]

@app.callback(
    [Output('dd-chunk', 'options'), Output('dd-chunk', 'value')],
    [Input('dd-file', 'value'), Input('dd-class', 'value')],
    State('input-dir', 'value')
)
def update_chunk_dropdown(fname, cls, base_dir):
    if not fname: return [], None
    path = Path(base_dir) / f'{cls}_Segments' / fname
    rate, raw = wavfile.read(path)
    n_chunks = max(1, len(raw) // T_ANALYSIS)
    opts = [{'label': f'Chunk {i + 1}', 'value': i} for i in range(n_chunks)]
    return opts, 0

@app.callback(
    [Output('graph-detail-scatter', 'figure'),
    Output('graph-detail-signal', 'figure'),
    Output('detail-info', 'children')],
    [Input('dd-chunk', 'value')],
    [State('dd-file', 'value'), State('dd-class', 'value'), State('input-dir', 'value')],
    prevent_initial_call=True
)
def update_detail_view(chunk_idx, fname, cls, base_dir):
    if fname is None or chunk_idx is None: return no_update

    path = Path(base_dir) / f'{cls}_Segments' / fname
    rate, raw = wavfile.read(path)
    sig = raw.astype(np.float64) - 128.0 if raw.dtype == np.uint8 else raw.astype(np.float64)
    smoothed = smooth_signal(sig, SMOOTHING_WINDOW)

    start = chunk_idx * T_ANALYSIS
    end = start + T_ANALYSIS
    chunk_sig = smoothed[start:end]

    points = extract_features(chunk_sig, 500, L_RANGE)

    fig_scatter = go.Figure()

```

```

if len(points) > 0:
    fig_scatter.add_trace(
        go.Scatter(x=points[:, 0], y=points[:, 1], mode='markers', name='Points', marker=dict(size=6,
opacity=0.5)))

centers, _ = find_3_density_centers(points)
info_text = [html.P(f"Знайдено точок: {len(points)}")]

if len(centers) == 3:
    c = np.array(centers)
    c_plot = np.vstack([c, c[0]])

    fig_scatter.add_trace(go.Scatter(x=c_plot[:, 0], y=c_plot[:, 1], mode='lines+markers', name='Triangle',
line=dict(color='yellow', width=2),
marker=dict(size=15, color='red', symbol='x')))

    Rs = calculate_R_distances(centers)
    info_text.append(html.P(f"Центри: {centers}"))
    info_text.append(html.P(f"Відстані R: {Rs[0]:.1f}, {Rs[1]:.1f}, {Rs[2]:.1f}"))
    info_text.append(html.P(f"Середнє R: {np.mean(Rs):.2f}"))

    fig_scatter.add_annotation(x=(c[0][0] + c[1][0]) / 2, y=(c[0][1] + c[1][1]) / 2, text=f"{Rs[0]:.1f}",
showarrow=False)
    fig_scatter.add_annotation(x=(c[0][0] + c[2][0]) / 2, y=(c[0][1] + c[2][1]) / 2, text=f"{Rs[1]:.1f}",
showarrow=False)
    fig_scatter.add_annotation(x=(c[1][0] + c[2][0]) / 2, y=(c[1][1] + c[2][1]) / 2, text=f"{Rs[2]:.1f}",
showarrow=False)
else:
    info_text.append(html.P("Не вдалося знайти 3 центри", className="text-danger"))

fig_scatter.update_layout(title="Трикутник голосних (Проміжні результати)", xaxis_title="V1",
yaxis_title="V2",
template="plotly_dark", xaxis_range=[0, 250], yaxis_range=[0, 250])

fig_signal = go.Figure(go.Scatter(y=chunk_sig))
fig_signal.update_layout(title="Сигнал (Часова область)", template="plotly_dark",
margin=dict(t=30, b=20, l=20, r=20))

return fig_scatter, fig_signal, info_text

if __name__ == '__main__':
    app.run(debug=True)

```